College Management System - API Documentation

Overview

The College Management System is a microservices-based enterprise application designed to manage academic, administrative, and library operations in a structured and scalable manner. The system is built using Spring Boot, Spring Cloud, Eureka Service Discovery, OpenFeign, JWT Authentication, and MySQL.

This documentation provides a complete overview of the architecture, service registry, corrected port mapping, authentication flow, service endpoints, and database structure. It also includes diagrams and screenshot placeholders for better presentation and project submission.

Table of Contents

- System Overview
- Key Features
- Technology Stack
- Architecture Overview
- Service Registry and Ports
- Base URLs
- Authentication and Security
- Service Communication Flow
- Database Design
- API Endpoints
- Screenshots
- Setup Instructions
- Conclusion

System Overview

The College Management System is designed to simplify and automate the management of students, faculty, departments, courses, subjects, attendance, library operations, and dashboards.

The project follows a **microservice architecture**, where each business module is separated into an independent service. This improves scalability, maintainability, fault isolation, and development flexibility.

Key Features

- Role-based access control for Admin, Faculty, Student, and Librarian.
- JWT-based secure authentication and authorization.
- Centralized API Gateway for request routing.
- Service discovery using Eureka Server.
- Inter-service communication using OpenFeign.
- Separate microservices for domain-specific responsibilities.
- MySQL-based persistent storage.
- Structured API documentation for all services.

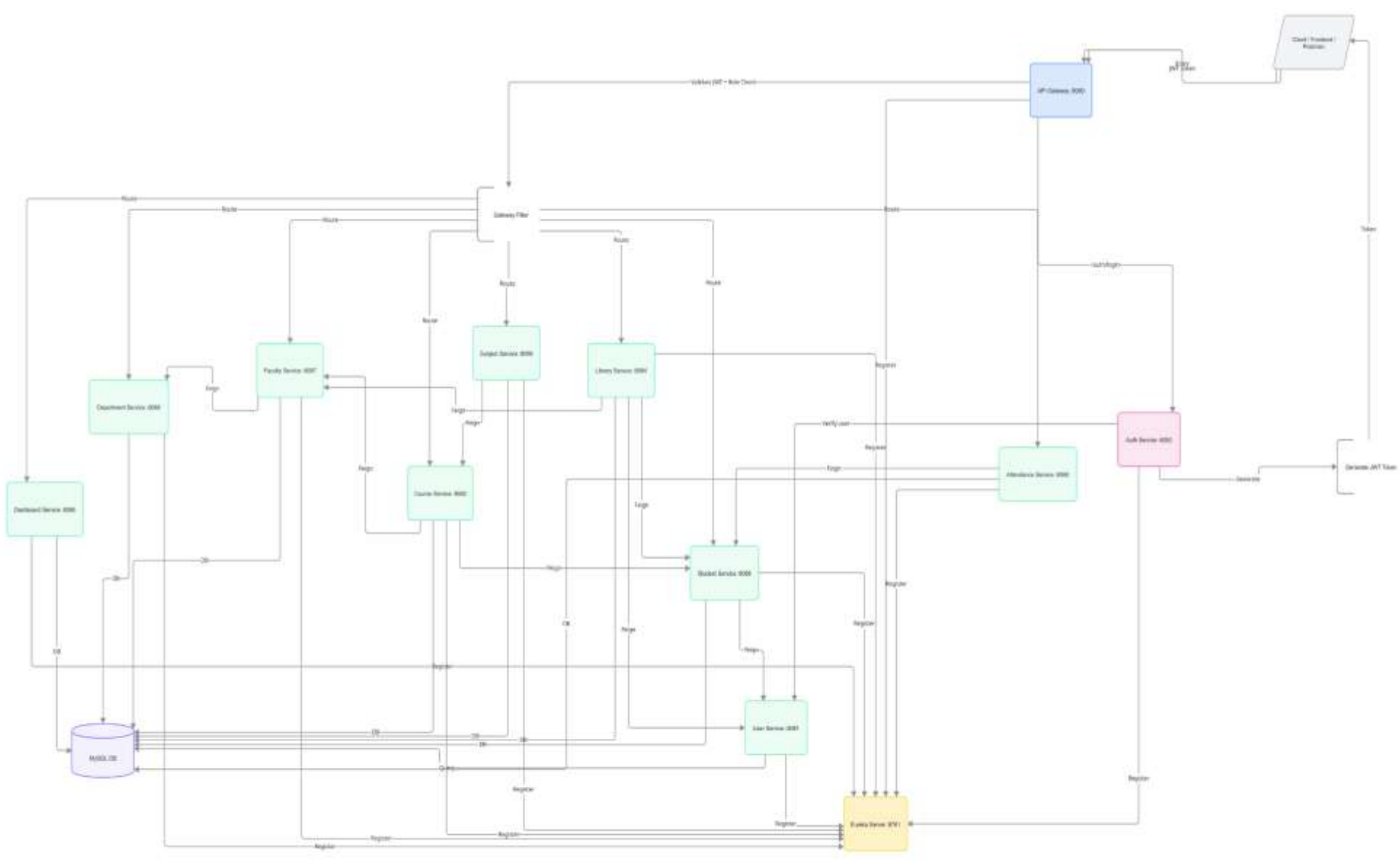
Technology Stack

Component	Technology
Backend Framework	Spring Boot
Microservices	Spring Cloud
API Gateway	Spring Cloud Gateway
Service Discovery	Eureka Server
Communication	OpenFeign
Security	JWT
Database	MySQL
API Documentation	Springdoc OpenAPI
Language	Java 17
ORM / Mapping	JPA / ModelMapper

Architecture Overview

The system follows a distributed microservices architecture. The client sends requests through the API Gateway, which routes them to the appropriate service. The services register with Eureka Server and communicate with each other when needed.

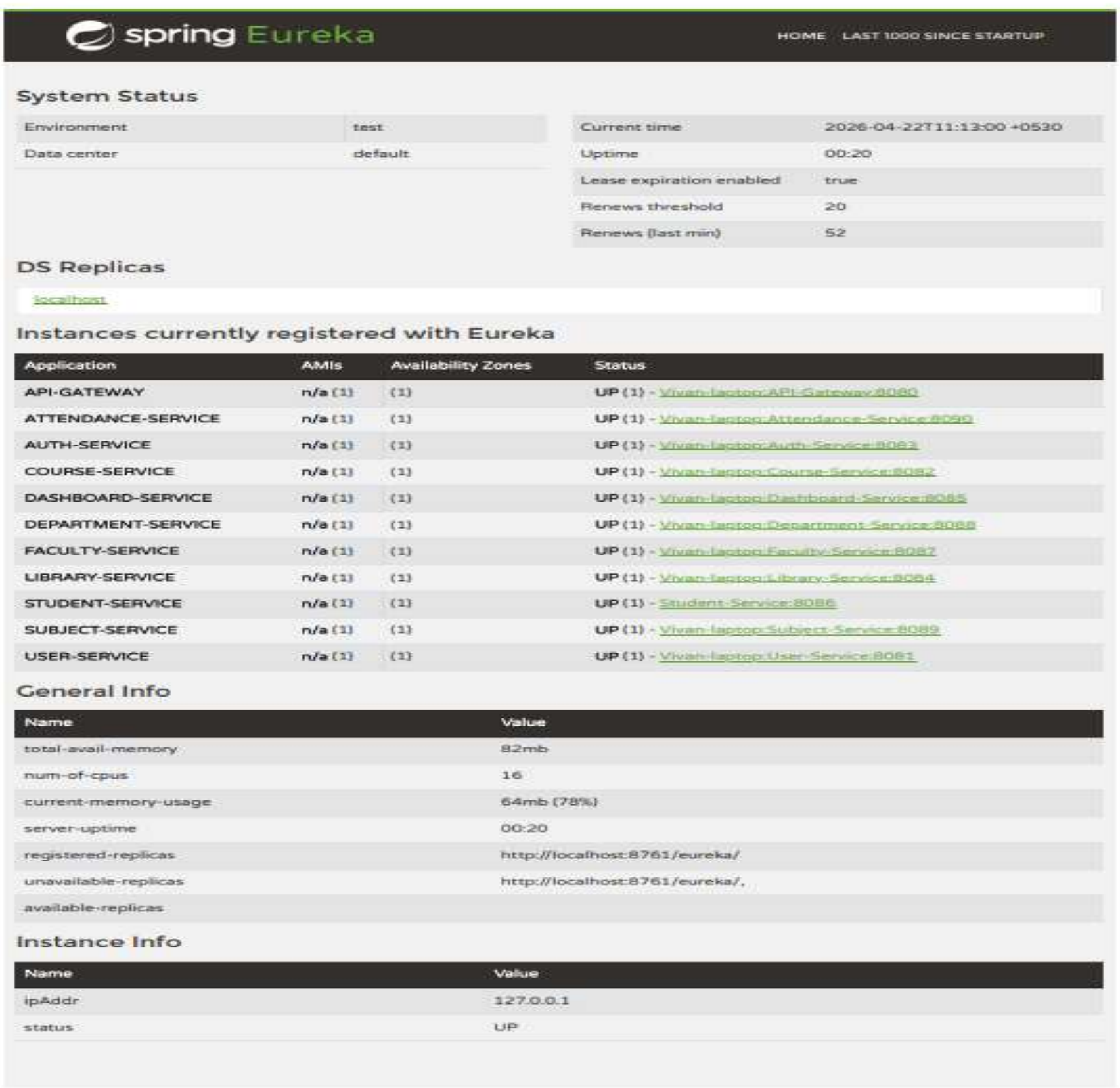
Architecture Flow



Service Registry and Ports

The following are the corrected service ports used in the system. These values should be reflected in both the application configuration and API documentation.

Service	Port
API Gateway	8080
Auth Service	8083
User Service	8081
Course Service	8082
Faculty Service	8087
Student Service	8086
Library Service	8084
Department Service	8088
Attendance Service	8090
Dashboard Service	8085
Subject Service	8089
Eureka Server	8761



Base URLs

Use the following base URLs in your API documentation and testing tools.

Service	Base URL
API Gateway	http://localhost:8080
Auth Service	http://localhost:8083/api/auth
User Service	http://localhost:8081/api/users
Course Service	http://localhost:8082/api/courses
Faculty Service	http://localhost:8087/api/faculty
Student Service	http://localhost:8086/api/students
Library Service	http://localhost:8084/api/library
Department Service	http://localhost:8088/api/departments
Attendance Service	http://localhost:8090/api/attendance
Dashboard Service	http://localhost:8085/api/dashboard
Subject Service	http://localhost:8089/api/subjects
Eureka Server	http://localhost:8761

Authentication and Security

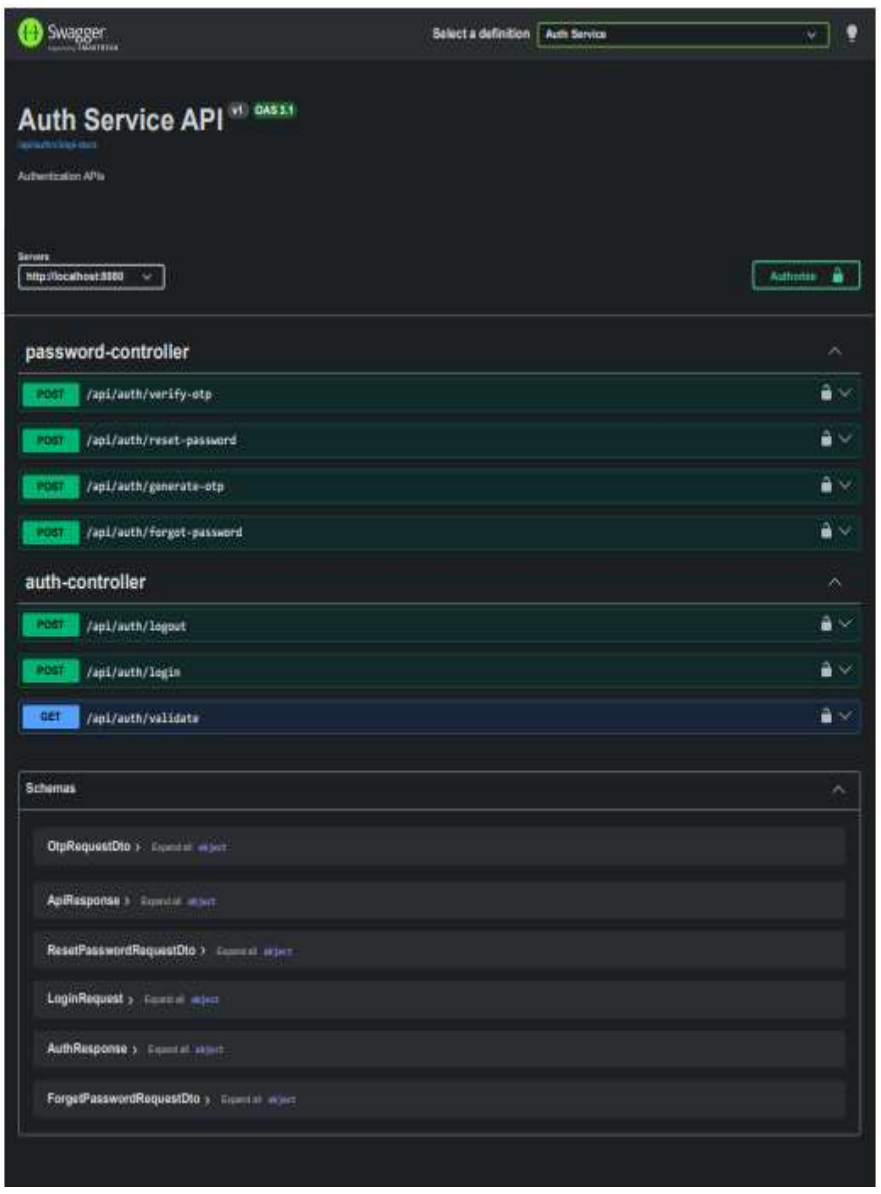
Authentication is handled through the Auth Service using JWT tokens. After successful login, the system generates a token that must be sent in the request header for protected endpoints.

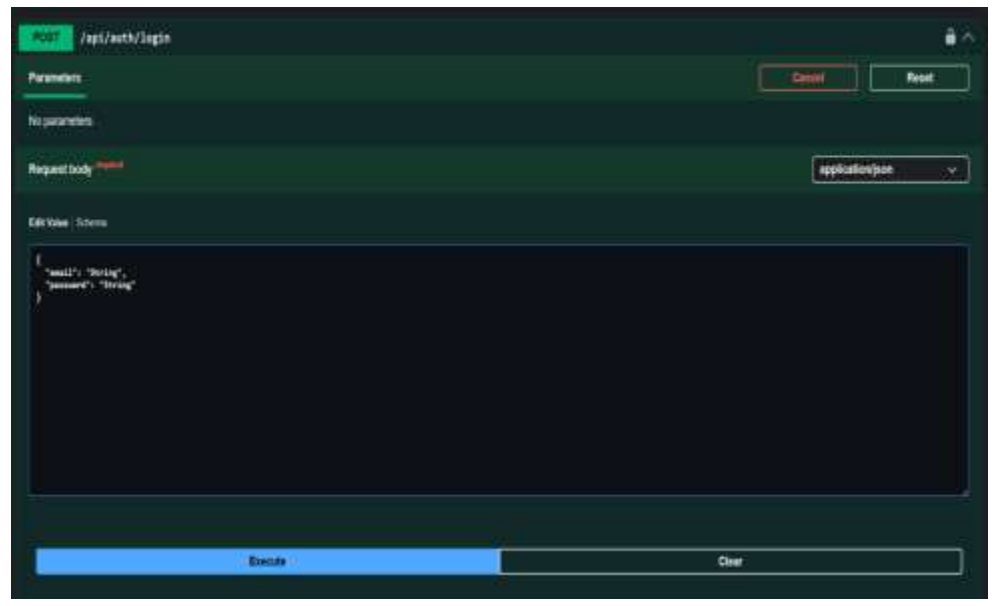
Security Highlights

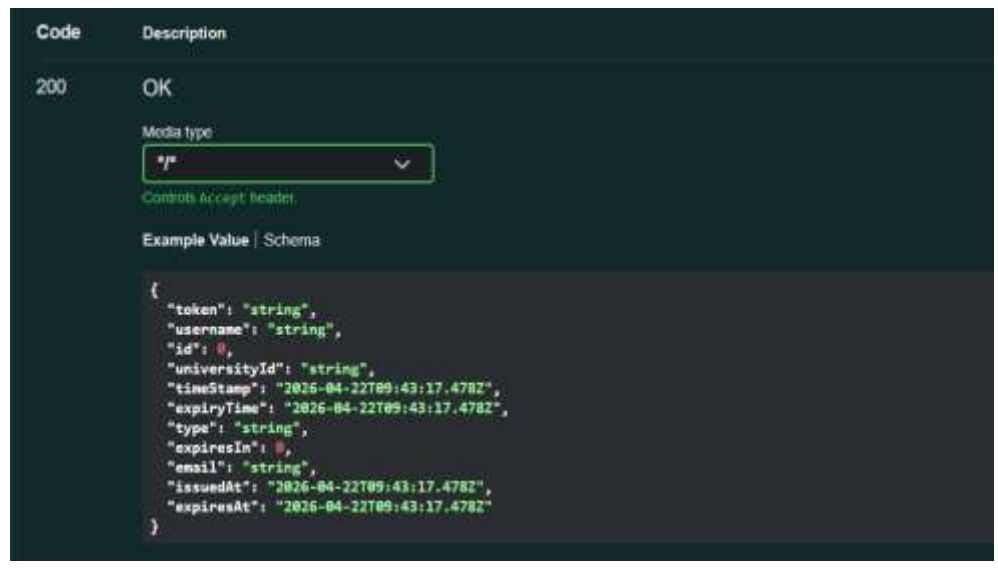
- User login is handled through the Auth Service.
- JWT token is generated after successful authentication.
- Protected APIs require Authorization: Bearer <token>.
- Tokens can be validated by the gateway or downstream services.
- Role-based authorization ensures users can access only permitted resources.

Authentication Flow

1. User sends credentials to the Auth Service.
2. Auth Service validates the credentials.
3. JWT token is generated.
4. Client includes the token in future requests.
5. Gateway forwards the request to the correct service.
6. Protected services validate the token before processing.





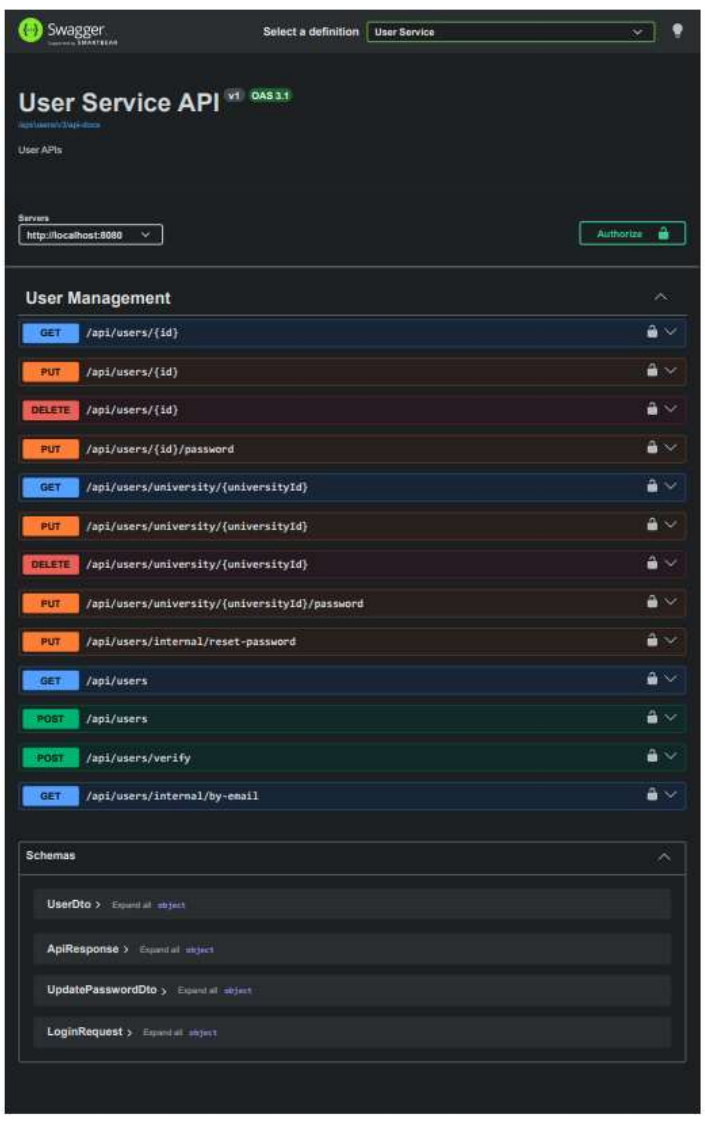


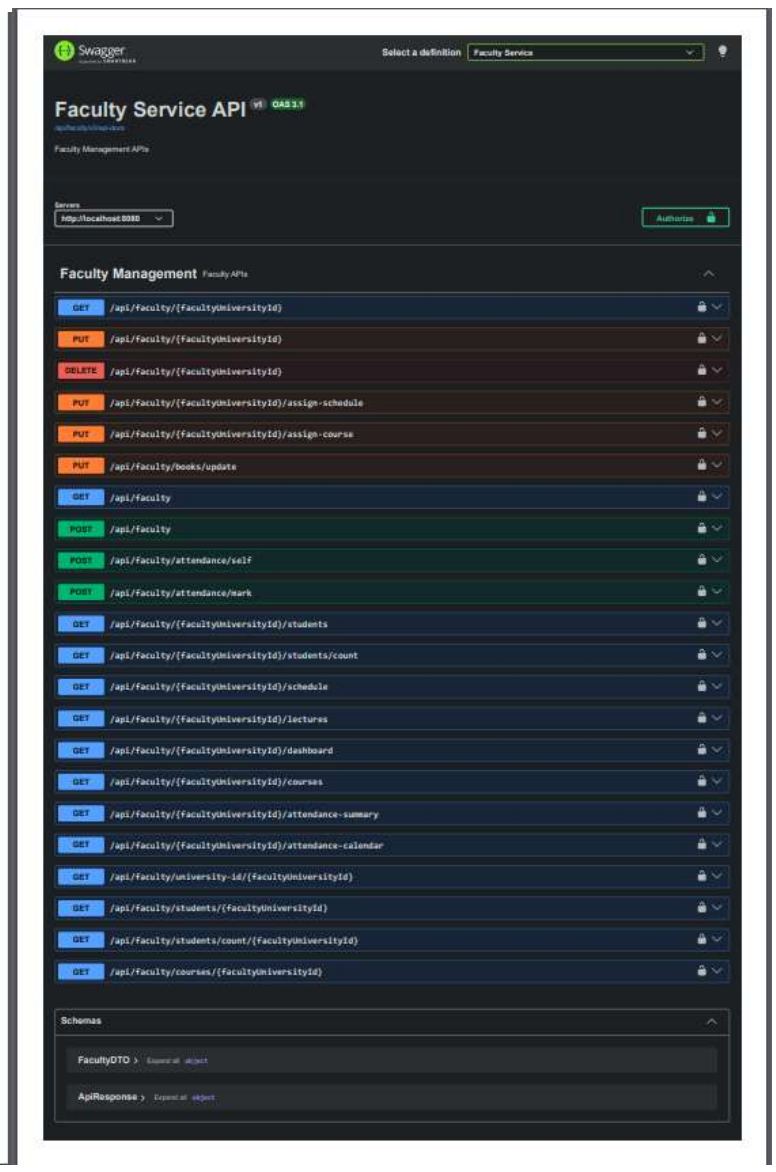
Service Communication Flow

Services communicate using OpenFeign for synchronous inter-service calls. For example, the Course Service can fetch faculty details, and the Library Service can resolve user-related information from other services when needed.

Key Communication Points

- API Gateway routes all client requests.
- Eureka provides dynamic service discovery.
- Auth Service manages authentication.
- OpenFeign is used for service-to-service communication.
- MySQL stores persistent data for each service module.

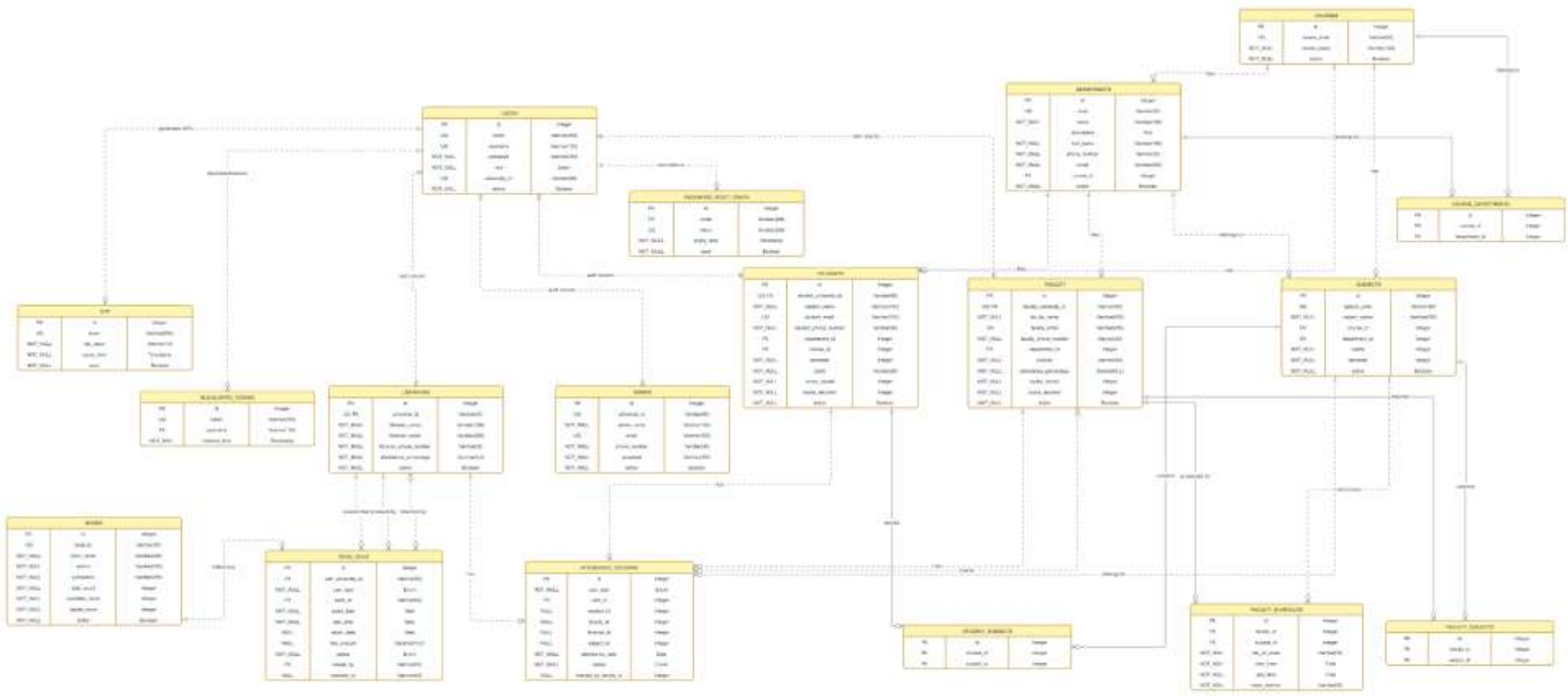




Database Design

The database structure is designed around the major domain entities of the college system. Each entity is stored in a separate table with relationships maintained through identifiers and business rules.

EER Diagram



Core Database Tables

Users

Stores authentication and profile details for all user types, including Admin, Faculty, Student, and Librarian.

Admins

Contains administrative user details for system-level management.

Faculty

Stores faculty profile, department, role, attendance, and related information.

Students

Stores student profile, course, semester, batch, and academic details.

Courses

Maintains course master records.

Subjects

Stores subject mapping with course, department, faculty, and student associations.

Departments

Contains department-level information and contact details.

Attendance Records

Tracks attendance of students by date, subject, and faculty.

Books

Manages the library inventory and book availability.

Librarians

Stores librarian profile and activity-related information.

Book Issue

Tracks book issuance, return, fine, and issue status.

Password Reset Token

Stores tokens used for password reset functionality.

OTP

Stores one-time passwords for verification flows.

Blacklisted Tokens

Stores invalidated JWT tokens for logout or security control.

API Endpoints

Add your actual endpoint list here based on each service.

Auth Service

- Register user
- Login user
- Refresh token
- Reset password

User Service

- Create user
- Get user by ID
- Get all users
- Update user
- Delete user

Course Service

- Create course
- Get course by code
- Get all courses
- Update course
- Delete course

Faculty Service

- Create faculty
- Get faculty details
- Assign course
- Update faculty
- Delete faculty

Student Service

- Create student
- Get student details
- Update student
- Delete student

Library Service

- Add book
- Issue book
- Return book
- Get library records

Department Service

- Create department
- Get department details
- Update department
- Delete department

Attendance Service

- Mark attendance
- Get attendance by student
- Get attendance by date
- Get attendance summary

Dashboard Service

- Get admin dashboard
- Get faculty dashboard
- Get student dashboard
- Get librarian dashboard

Subject Service

- Create subject
- Assign subject
- Get subject details
- Update subject
- Delete subject

Setup Instructions

Prerequisites

- Java 17 or above
- Maven
- MySQL 8+
- IntelliJ IDEA or any Java IDE
- Postman
- Git

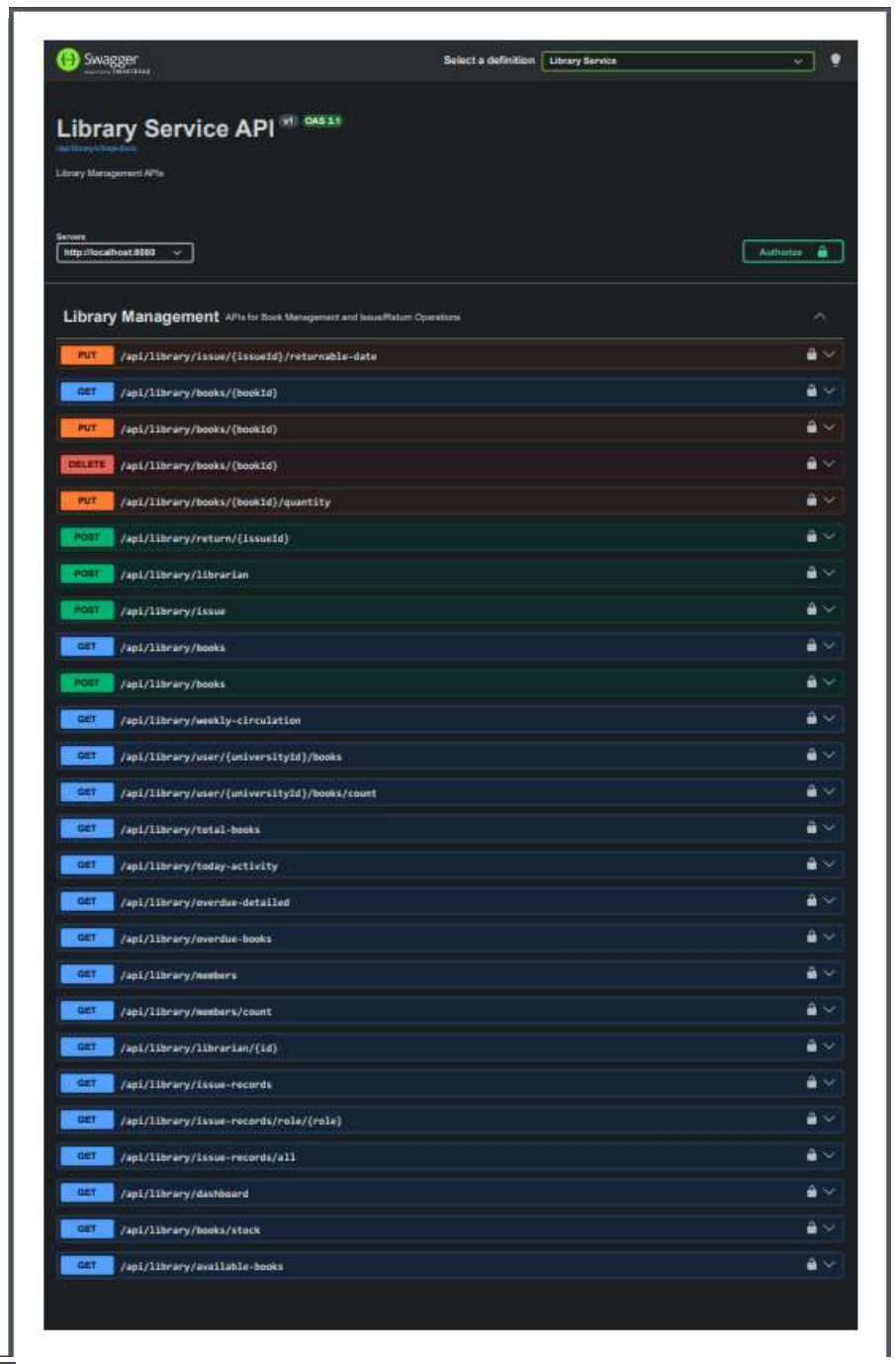
Run Order

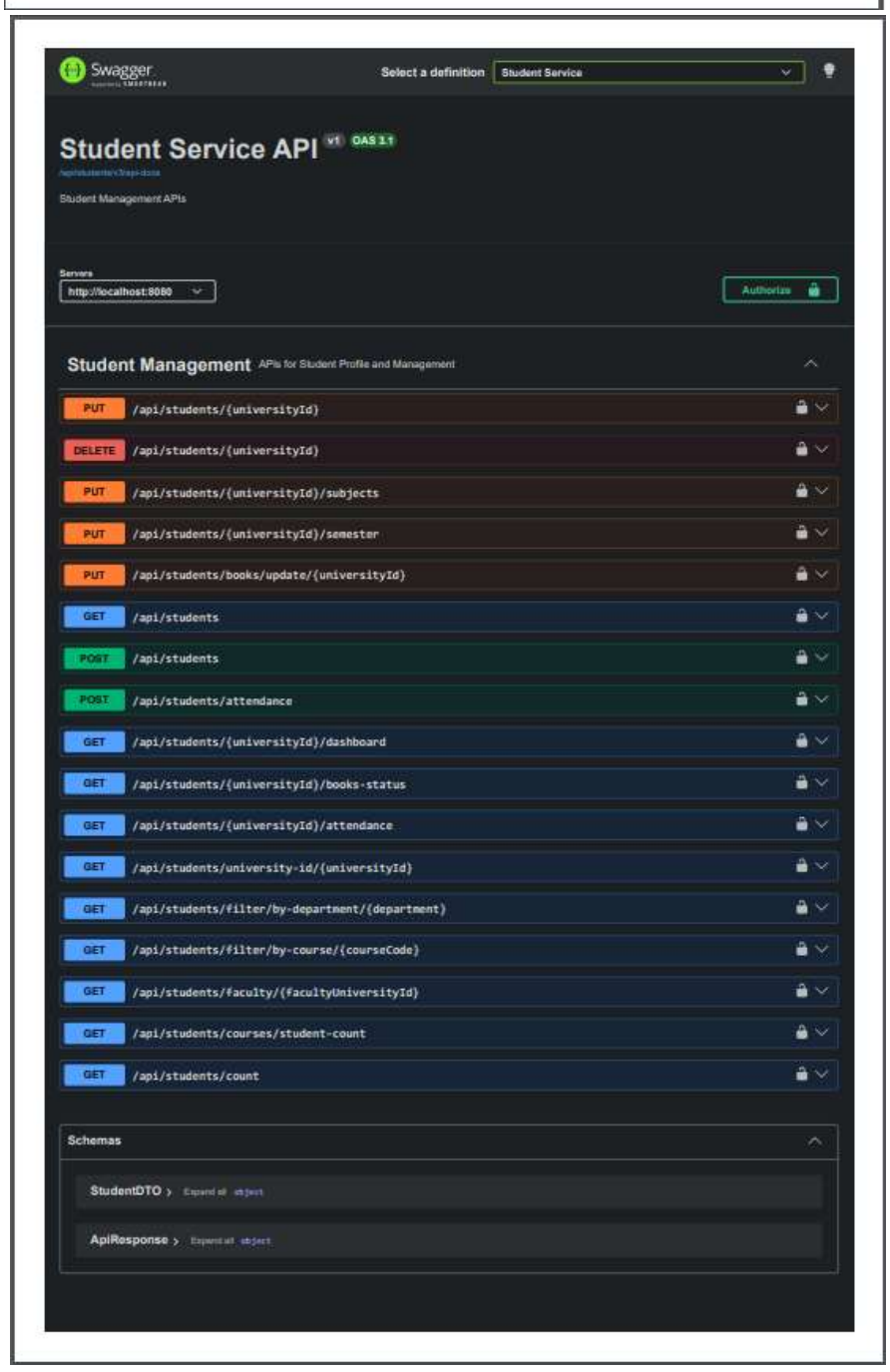
1. Start MySQL server.
2. Start Eureka Server.
3. Start Auth Service.
4. Start User Service.
5. Start all remaining microservices.
6. Start API Gateway.
7. Open Swagger UI or test APIs through Postman.

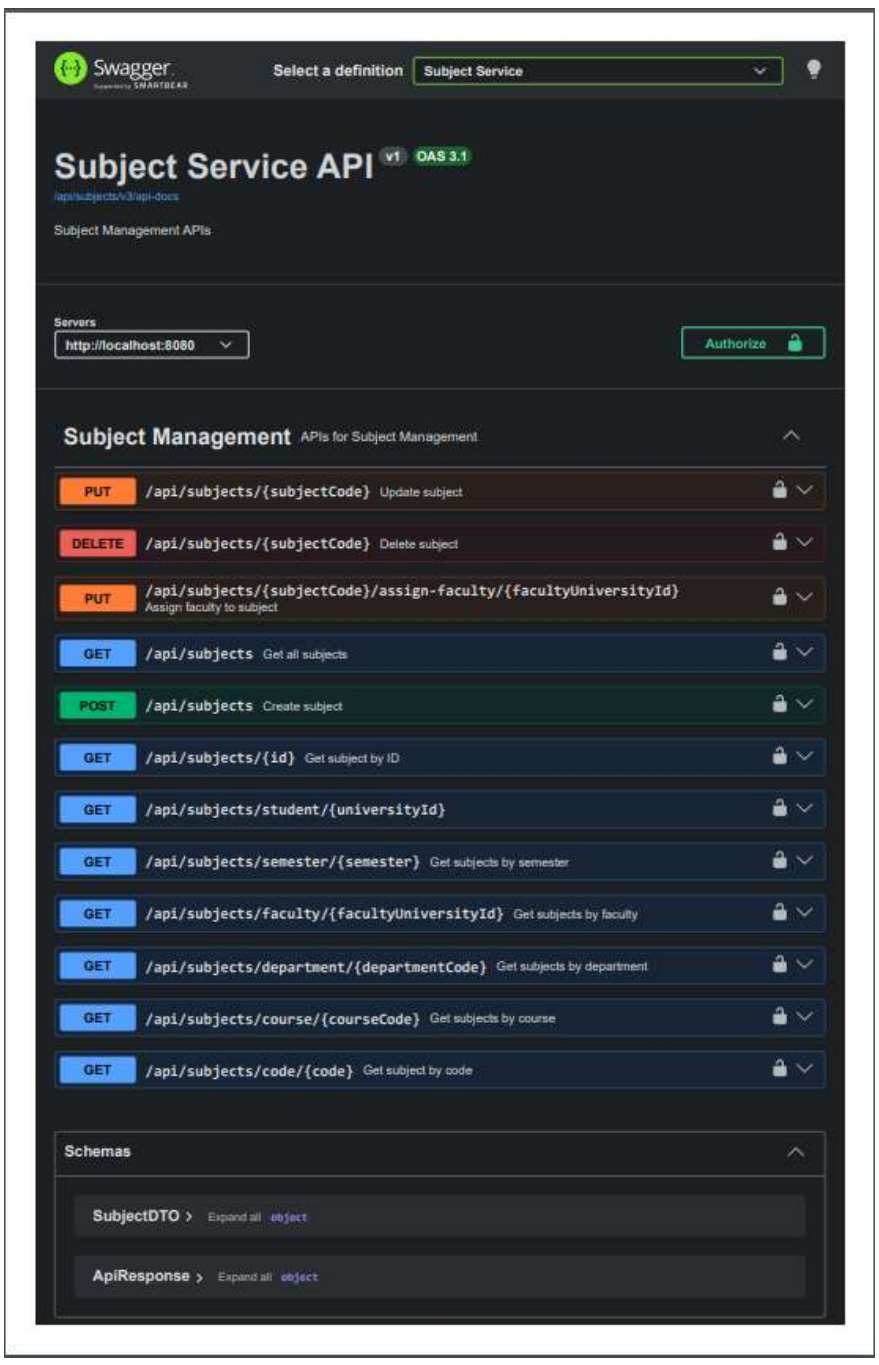
Application Access

- Eureka Server: http://localhost:8761
- API Gateway: http://localhost:8080
- Swagger UI: available per service
- MySQL: local database instance

Screenshots







Swagger Select a definition Attendance Service

Attendance Service API v1 OAS 3.1

/api/attendance/v3/api-docs

Attendance Management APIs

Servers http://localhost:8080 Authorize

Attendance Management APIs for Attendance Management

- GET /api/attendance/{id} Get attendance record
- PUT /api/attendance/{id} Update attendance
- DELETE /api/attendance/{id} Delete attendance
- POST /api/attendance Mark attendance
- GET /api/attendance/student/{studentId} Get student attendance
- GET /api/attendance/percentage/{studentId}/range Get attendance percentage
- GET /api/attendance/faculty/{facultyId}/range Get faculty attendance records
- GET /api/attendance/course/{courseId}/date/{date} Get course attendance for date

Schemas

AttendanceDTO > Expand all object

Swagger Select a definition Course Service

Course Service API v1 OAS 3.1

/api/courses/v3/api-docs

Course Management APIs

Servers http://localhost:8080 Authorize

course-controller

- PUT /api/courses/{code}
- DELETE /api/courses/{code}
- GET /api/courses
- POST /api/courses
- POST /api/courses/assign/faculty/{facultyUniversityId}
- GET /api/courses/{id}
- GET /api/courses/faculty/{facultyUniversityId}
- GET /api/courses/count

Schemas

CourseRequestDto > Expand all object

ApiResponse > Expand all object

Swagger Select a definition Dashboard Service

Dashboard Service API v1 OAS 3.1

/api/dashboard/v3/api-docs

Dashboard and Analytics APIs

Servers http://localhost:8080 Authorize

Dashboard APIs Role-based dashboard endpoints

- GET /api/dashboard/student/{universityId} Student dashboard
- GET /api/dashboard/role/{role}/{userId} Generic role-based dashboard (use specific endpoints preferred)
- GET /api/dashboard/librarian Librarian dashboard
- GET /api/dashboard/faculty/{facultyUniversityId} Faculty dashboard
- GET /api/dashboard/admin Admin dashboard statistics

profile-controller

- GET /api/profile/me

Schemas

ApiResponse > Expand all object

Swagger Select a definition Department Service

Department Service API v1 OAS 3.1

/api/departments/v3/api-docs

Department Management APIs

Servers http://localhost:8080 Authorize

Department Management APIs for Department Management

- GET /api/departments/{id}
- PUT /api/departments/{id}
- DELETE /api/departments/{id}
- GET /api/departments
- POST /api/departments
- GET /api/departments/count/total
- GET /api/departments/code/{code}

Schemas

DepartmentDTO > Expand all object

ApiResponse > Expand all object

Schemas

SubjectDTO ^ Collapse all object

- id integer int64
- subjectCode* string ≥ 1 characters
- subjectName* string ≥ 1 characters
- courseId* string ≥ 1 characters
- courseCode* string ≥ 1 characters
- courseName string
- departmentId* string ≥ 1 characters
- departmentCode* string ≥ 1 characters
- credits integer ≥ 1 int32
- semester integer ≥ 1 int32
- studentUniversityId string
- description string
- courseObjectives string
- outcomes string
- facultyId string
- facultyName string
- totalStudentsEnrolled integer int32
- active boolean
- createdAt string date-time
- updatedAt string date-time
- createdBy string
- updatedBy string

ApiResponse ^ Collapse all object

- message string
- status integer int32
- data any
- timestamp string date-time

Schemas

StudentDTO ^ Collapse all object

- id integer int64
- universityId string
- studentName string
- studentEmail string
- studentPhoneNumber string
- semester string
- batch string
- department string
- course string
- courseCode string
- facultyUniversityId string
- facultyName string
- booksIssued integer int32
- booksReturned integer int32
- attendancePercentage number float
- active boolean
- createdAt string date-time
- updatedAt string date-time
- createdBy string
- updatedBy string
- password string
- subjects ^ Collapse all array<string>
  - Items string

ApiResponse ^ Collapse all object

- message string
- status integer int32
- data any
- timestamp string date-time

Schemas

ApiResponse ^ Collapse all object

- message string
- status integer int32
- data any
- timestamp string date-time

BookDTO ^ Collapse all object

- id integer int64
- bookId string
- bookName string
- author string
- publication string
- totalCount integer int32
- availableCount integer int32
- issuedCount integer int32
- description string
- active boolean
- createdAt string date-time
- updatedAt string date-time

LibrarianDTO ^ Collapse all object

- id integer int64
- universityId string
- librarianName string
- librarianEmail string
- librarianPhoneNumber string
- attendanceCalendar ^ Collapse all object
  - Additional properties any
- attendancePercentage number float
- active boolean
- createdAt string date-time
- updatedAt string date-time
- createdBy string
- updatedBy string

BookIssueDTO ^ Collapse all object

- id integer int64
- bookId string
- bookName string
- userId string
- userName string
- userEmail string
- department string
- course string
- userRole string
- issueDate string date
- returnableDate string date
- returnedDate string date
- activeIssue boolean
- status ^ Collapse all string
  - Enum ^ Collapse all array
    - #0="ISSUED"
    - #1="RETURNED"

Schemas

FacultyDTO ^ Collapse all object

- id integer int64
- facultyUniversityId string
- facultyName string
- facultyEmail string
- username string
- password string
- facultyPhoneNumber string
- department string
- subRole ^ Collapse all string
  - Enum ^ Collapse all array
    - #0="HOD"
    - #1="PROFESSOR"
    - #2="ASSISTANT_PROFESSOR"
    - #3="TRAINEE"
- courseCode ^ Collapse all array<string>
  - Items string
- totalCourse integer int32
- attendancePercentage number float
- booksIssued integer int32
- booksReturned integer int32
- attendanceCalendar ^ Collapse all object
  - Additional properties any
- scheduleData ^ Collapse all object
  - Additional properties any
- active boolean
- createdAt string date-time
- updatedAt string date-time
- createdBy string
- updatedBy string

ApiResponse ^ Collapse all object

- message string
- status integer int32
- data any
- timestamp string date-time

Schemas

DepartmentDTO Collapse all object

- id integer int64
- departmentName* string ≥ 1 characters
- departmentCode* string ≥ 1 characters
- hodId string
- hodName string
- totalFaculty integer int32
- totalStudents integer int32
- description string
- active boolean
- createdAt string date-time
- updatedAt string date-time
- createdBy string
- updatedBy string

ApiResponse Collapse all object

- message string
- status integer int32
- data any
- timestamp string date-time

Schemas

ApiResponse Collapse all object

- message string
- status integer int32
- data any
- timestamp string date-time

Schemas

CourseRequestDto Collapse all object

- name* string ≥ 1 characters
- courseCode* string ≥ 1 characters

ApiResponse Collapse all object

- message string
- status integer int32
- data any
- timestamp string date-time

Conclusion

The College Management System demonstrates a clean microservices-based approach for managing a complete college ecosystem. With corrected service ports, structured endpoints, secure authentication, and service discovery, the application is now easier to maintain, test, and extend.